Design and Implementation of a Variable Declaration Compiler

A PROJECT REPORT

submitted by

1. **Nancy Sunil – AJC24CS150**
2. **Naveena Ruby Thomas – AJC24CS153**
3. **Pavan V Anish – AJC24CS165**
4. **Ronal Sajees – AJC24CS178**
5. **Sanjith Babu – AJC24CS191**
6. **Shawn Abraham – AJC24CS198**

to

the APJ Abdul Kalam Technological University
in partial fulfillment of the requirement for the award of the Degree of

Bachelor of Technology

in

Computer Science and Engineering

Under the guidance of
Ms. TINTU ALPHONSA THOMAS
Assistant Professor

Department of Computer Science and Engineering Amal Jyothi College of Engineering(Autonomous) Kanjirappally-686518

October 2026

TABLE OF CONTENTS

1.	Abstract	3
2.	Introduction and Problem Statement	4
3.	Language Specification and Grammar	6
4.	System Design	8
5.	Implementation	11
6.	Testing and Results	15
7.	Project Structure and Documentation	21
8.	Individual Contributions	23
9.	Limitations and Future Enhancements	24
10.	Conclusion	25

ABSTRACT

A compiler is a software system that processes source code written in a programming language and checks whether it follows the rules of that language. Compiler construction involves several stages, including lexical analysis, syntax analysis, abstract syntax tree construction, and semantic analysis.

The project entitled “Design and Implementation of a Variable Declaration Compiler” focuses on implementing the fundamental stages of a compiler front end for a simplified programming language. The language is designed to support variable declarations, optional initialization, and assignment statements using basic data types such as int, float, double, char, and bool.

The lexical analyzer identifies tokens such as keywords, identifiers, numeric literals, character literals, Boolean literals, operators, and punctuation symbols. The parser checks the grammatical structure of the token sequence and constructs an Abstract Syntax Tree (AST) representing the program. The AST provides a structured representation of declarations and assignments. A semantic analyzer can validate properties such as variable declaration rules and type compatibility, depending on the implemented checks.

The project demonstrates how formal grammar, tokenization, parsing, structured program representation, and semantic validation work together in compiler construction. It also provides a foundation for extending the language with additional constructs and more advanced compiler features.

CHAPTER 1: INTRODUCTION

1.1 Background

Compiler design is an important area of computer science that deals with translating and validating programs written in programming languages. A compiler processes source code through different stages to determine whether it follows the language's rules and to prepare it for further processing.

The major stages of a compiler front end include lexical analysis, syntax analysis, abstract syntax tree construction, and semantic analysis. Lexical analysis identifies meaningful tokens in the source program. Syntax analysis checks whether these tokens follow the grammar of the language. AST construction represents the program in a structured form, while semantic analysis checks rules related to the meaning of the program.

Studying these stages through a practical implementation helps in understanding how programming languages are designed and processed.

1.2 Problem Statement

The theoretical study of compiler design may not fully demonstrate how individual compiler stages interact during program processing. A practical system is needed to illustrate how source code containing variable declarations and assignments can be tokenized, parsed, represented as a structured tree, and checked for correctness.

The project addresses this requirement by developing a compiler front end for a simplified programming language that supports basic variable declarations and assignment statements.

1.3 Objectives

1. To design a simplified programming language for variable declarations and assignments.
2. To define a Context-Free Grammar (CFG) for the supported language constructs.
3. To implement a lexical analyzer that identifies tokens from source code.
4. To develop a parser that validates the syntactic structure of a program.
5. To construct an Abstract Syntax Tree representing declarations and assignments.

6. To implement semantic checks for variable declarations, assignments, and data types, where supported.
7. To identify and report invalid source programs through suitable error handling.
8. To integrate the compiler components into an application, if included in the final implementation.
9. To test the compiler using valid and invalid input programs.

1.4 Scope of the Project

The project focuses on the fundamental stages of compiler front-end development. Its intended scope includes variable declarations, optional initialization, assignment statements, basic data types, lexical analysis, syntax analysis, and AST generation.

The data types considered are int, float, double, char, and bool. The language uses identifiers, literals, assignment operators, and statement terminators.

Advanced constructs such as functions, loops, conditional statements, complex expressions, and machine-code generation are outside the current scope unless they have been implemented.

1.5 Proposed System

The proposed system processes source code through a series of compiler stages. The lexer converts the input into tokens, and the parser checks whether the tokens follow the specified grammar. If parsing succeeds, the parser generates an AST representing the program's statements. The semantic analyzer can then perform additional checks on the resulting structure.

The system is organized into separate modules to improve readability, maintainability, and testing. This modular design allows individual components to be developed and verified independently.

CHAPTER 2: LANGUAGE SPECIFICATION AND GRAMMAR

2.1 Language Overview

The Variable Declaration Compiler processes a simplified, C-like programming language. The language supports variable declarations, declarations with initialization, and assignment statements. Each statement is terminated by a semicolon.

Examples of the intended language constructs are:

```
int age;
int age = 20;
float mark = 95.5;
char grade = 'A';
bool passed = true;
age = 21;
```

These examples illustrate the intended language. Their acceptance by the final compiler must be verified through testing.

2.2 Lexical Elements

Lexical elements are the smallest meaningful units identified by the lexical analyzer.

Keywords: Reserved words used to specify data types, such as int, float, double, char, and bool.

Identifiers: Names assigned to variables, such as age, mark, grade, and passed.

Numeric literals: Numeric constants such as 20 and 95.5.

Character literals: Individual characters enclosed in single quotation marks, such as 'A'.

Boolean literals: Logical values such as true and false.

Assignment operator: The symbol =, used to assign a value to a variable.

Statement terminator: The semicolon (;), which marks the end of a statement.

The exact token names and lexical rules depend on the implementation of the lexer.

2.3 Context-Free Grammar

A Context-Free Grammar (CFG) defines the syntactic structure of a programming language. It consists of terminals, non-terminals, production rules, and a start symbol.

Based on the parser structure discussed during development, the following is a provisional grammar for the project:

```
Start Symbol: Program
Program → StatementList
StatementList → Statement StatementList | ε
Statement → Declaration | Assignment
Declaration → Type ID ;
Declaration → Type ID = Value ;
Assignment → ID = Value ;
Type → int | float | double | char | bool
Value → ID | NUMBER | CHAR_LITERAL | BOOL_LITERAL
```

Here, ID represents an identifier, NUMBER represents a numeric literal, CHAR_LITERAL represents a character literal, BOOL_LITERAL represents a Boolean literal, and ε represents the empty production.

2.4 Grammar Transformations

Grammar transformations are used to modify a grammar while preserving the language it generates or to make the grammar more suitable for a particular parsing technique.

```
StatementList → Statement StatementList | ε
```

This production represents a sequence of statements and allows the statement list to terminate through the empty production. It does not exhibit immediate left recursion because StatementList does not appear as the leftmost symbol on the right-hand side of its recursive production.

2.5 FIRST and FOLLOW Sets

FIRST and FOLLOW sets are important concepts in predictive parsing. The FIRST set of a grammar symbol contains the terminals that can begin strings derived from that symbol. The FOLLOW set of a non-terminal contains the terminals that can appear immediately after that non-terminal in a valid sentential form.

For the implemented grammar, FIRST and FOLLOW sets can be calculated to assist in analyzing parsing decisions and determining whether the grammar is suitable for predictive parsing.

CHAPTER 3: SYSTEM DESIGN

3.1 System Architecture

The system is designed around the major stages of a compiler front end. The logical processing sequence is:

Source Code Input → Lexical Analysis → Syntax Analysis → AST Construction → Semantic Analysis → Output

Source Code Input: The user provides a program containing variable declarations and assignment statements.

Lexical Analysis: The lexer identifies tokens and provides them to the parser.

Syntax Analysis: The parser checks the token sequence against the grammar.

AST Construction: The parser constructs a structured representation of the program.

Semantic Analysis: The semantic analyzer performs meaning-related checks, where implemented.

Output: The application presents the AST, validation results, or errors according to its actual functionality.

3.2 Module Description

3.2.1 Lexical Analyzer

The lexical analyzer reads the source code and converts it into tokens. It recognizes supported keywords, identifiers, literals, operators, and punctuation symbols. The token stream serves as the input to the parser. Token positions, including line and column numbers, may also be maintained to help identify the location of errors.

3.2.2 Grammar Module

The grammar module contains the production rules defining the structure of valid programs. It specifies how programs, statement lists, declarations, assignments, types, and values can be formed. Separating the grammar from the parser helps keep the language specification organized and makes the grammar easier to inspect and modify.

3.2.3 Parser

The parser processes the token sequence produced by the lexer. It checks whether the sequence follows the grammar and constructs AST nodes for valid statements. The current parser design includes structures for Program, Declaration, Assignment, and Value. It uses a recursive-descent approach, according to the parser implementation discussed during development.

The parser should also handle invalid input, such as missing semicolons or incomplete declarations, through its error-handling mechanism.

3.2.4 Abstract Syntax Tree

An Abstract Syntax Tree is a hierarchical representation of the essential structure of a program. Unlike a flat token sequence, the AST groups related elements into meaningful nodes.

Program: Represents the complete program and contains its statements.

Declaration: Represents a variable declaration, including its type, identifier, and optional initial value.

Assignment: Represents an assignment statement, including the target identifier and assigned value.

Value: Represents a literal or identifier used in a declaration or assignment.

The AST can be converted into a dictionary structure and serialized as JSON for inspection and further processing.

3.2.5 Semantic Analyzer

The semantic analyzer checks whether statements satisfy meaning-related rules that cannot be verified by syntax analysis alone. Possible checks include detecting the use of undeclared variables, duplicate declarations, incompatible assignments, and assignments to variables that have not been declared. Only checks actually implemented in the semantic analyzer should be reported as completed features.

3.3 AST Design

Consider the following source program:

```
int age = 20;
age = 21;
```

The corresponding AST can be represented hierarchically as follows:

```
Program
|-- Declaration
|   |-- Type: int
|   |-- Identifier: age
|   |-- Value: 20
|-- Assignment
    |-- Identifier: age
    |-- Value: 21
```

The Program node is the root of the tree. It contains a declaration node and an assignment node. The declaration node stores the data type, identifier, and initial value, while the assignment node stores the identifier and the new value.

3.4 Data Flow

1. The source code is supplied to the application.
2. The lexer converts the source code into tokens.
3. The parser checks the syntax and constructs the AST.
4. The AST is passed to the semantic analyzer, if connected in the implemented pipeline.
5. The compiler returns the appropriate result or error information.
6. The application displays the result to the user, if a user interface is implemented.

This separation of responsibilities makes the compiler easier to test and maintain.

CHAPTER 4: IMPLEMENTATION

4.1 Technologies Used

Python: Used to implement the compiler logic.

Lexer module: Responsible for identifying tokens from the source program.

Grammar module: Defines the production rules for the supported language.

Parser module: Performs syntax analysis and constructs the AST.

Semantic analyzer module: Responsible for semantic validation, if implemented.

Flask: Used for the backend if the application is built as a Flask web application.

HTML, CSS, and JavaScript: Used for the frontend if the project includes a web interface.

4.2 Lexical Analyzer Implementation

The lexer scans the source program and groups characters into tokens. For example, the statement below can conceptually be represented by a token sequence.

```
int age = 20;
KEYWORD(int)
IDENTIFIER(age)
ASSIGN(=)
NUMBER(20)
SEMICOLON(;)
```

Each token may contain its category, lexeme, line number, and column number. This information is useful for syntax analysis and error reporting. The lexer must distinguish reserved keywords from identifiers and recognize literals according to the lexical rules of the language.

4.3 Grammar Implementation

The grammar.py module defines the grammar used to describe valid source programs. The grammar represents a program as a statement list. Each statement is a declaration or assignment. A declaration specifies a data type and identifier and may optionally include an initial value.

The parser uses these rules to determine whether the token sequence represents a valid program. Keeping the grammar in a separate module improves clarity and makes it easier to modify the language specification without mixing it with unrelated application logic.

```
CONTEXT-FREE GRAMMAR (CFG)

Program -> StatementList
StatementList -> Statement StatementList | ε
Statement -> Declaration | Assignment
Declaration -> Type ID ; | Type ID = Value ;
Assignment -> ID = Value ;
Type -> int | float | double | char | bool
Value -> ID | NUMBER | CHAR_LITERAL | BOOL_LITERAL

FIRST SETS
FIRST(Program) = {'int', 'ε', 'bool', 'ID', 'float', 'double', 'char'}
FIRST(StatementList) = {'int', 'ε', 'bool', 'ID', 'float', 'double', 'char'}
FIRST(Statement) = {'float', 'int', 'double', 'char', 'bool', 'ID'}
FIRST(Declaration) = {'float', 'int', 'double', 'char', 'bool'}
FIRST(Assignment) = {'ID'}
FIRST(Type) = {'float', 'int', 'double', 'char', 'bool'}
FIRST(Value) = {'CHAR_LITERAL', 'NUMBER', 'ID', 'BOOL_LITERAL'}

FOLLOW SETS
FOLLOW(Program) = {'$'}
FOLLOW(StatementList) = {'$'}
FOLLOW(Statement) = {'int', '$', 'bool', 'ID', 'float', 'double', 'char'}
FOLLOW(Declaration) = {'int', '$', 'bool', 'ID', 'float', 'double', 'char'}
FOLLOW(Assignment) = {'int', '$', 'bool', 'ID', 'float', 'double', 'char'}
FOLLOW(Type) = {'ID'}
FOLLOW(Value) = {';'}
```

4.4 Parser Implementation

The parser consumes the tokens produced by the lexer and verifies their structure according to the grammar. The parser contains AST node classes representing Program, Declaration, Assignment, and Value. The parse() function returns a Program object, while parse_source() provides a source-level parsing interface when the lexer is available.

The parser constructs a node for each valid statement and organizes these nodes under the Program root. Syntax errors should be reported when the source program does not follow the grammar. Examples include incomplete declarations, missing statement terminators, and invalid statement structures.

4.5 AST Construction

AST construction is performed during syntax analysis. The parser creates objects corresponding to the structures recognized in the source program.

Consider the following input:

```
int age = 20;
float mark = 95.5;
age = 21;
```

The AST contains three statement nodes: two declaration nodes and one assignment node. The Declaration node stores the declared type, identifier, and optional initial value. The Assignment node stores the target identifier and assigned value. The Value node stores information about the literal or identifier.

The to_dict() methods convert the AST objects into dictionary representations, which can then be serialized into JSON. This representation is useful for debugging, testing, and displaying the program structure.

4.6 Semantic Analysis Implementation

Semantic analysis checks whether a syntactically valid program follows the semantic rules of the language.

For example, the following program may be valid if age has been declared and the assigned value is compatible with its type:

```
int age = 20;
age = 21;
```

However, assigning to a variable that has not been declared may be invalid:

```
height = 180;
```

An assignment involving incompatible data types may also be rejected according to the language's type rules. The semantic analyzer may use a symbol table or another data structure to track variable declarations and associated types.

4.7 Application Integration

If the project includes a web interface, the frontend allows users to enter source code and submit it for analysis. The backend receives the input, invokes the compiler components, and returns the output.

Depending on the implemented interface, the output may include tokens, an AST in JSON format, semantic-analysis results, or error messages.



CHAPTER 5: TESTING AND RESULTS

5.1 Testing Objectives

Testing verifies whether the individual components and the complete system behave according to the language specification.

7. To verify that the lexer recognizes valid tokens.
8. To check whether the parser accepts valid programs.
9. To verify AST construction for declarations and assignments.
10. To check the handling of invalid syntax.
11. To evaluate semantic checks, where implemented.
12. To verify communication between compiler modules and the application interface, if applicable.

5.2 Test Cases

The following test cases are proposed for evaluating the compiler. The status must be updated after running each test on the actual implementation.

Test ID	Input / Condition	Expected Result	Status
TC01	int age = 20;	Declaration recognized and AST constructed	To be tested
TC02	float mark = 95.5;	Float declaration recognized	To be tested
TC03	char grade = 'A';	Character declaration recognized	To be tested
TC04	bool passed = true;	Boolean declaration recognized	To be tested
TC05	int age = 20; age = 21;	Declaration and assignment represented in AST	To be tested
TC06	int age = 20 (without semicolon)	Syntax error if handled by parser	To be tested
TC07	height = 180;	Undeclared-variable error if implemented	To be tested
TC08	int age = 20; int age = 21;	Duplicate-declaration error if implemented	To be tested

TC09	int age = 20; age = 21.5;	Type validation according to implemented rules	To be tested
TC10	Empty source program	Accepted or rejected according to grammar and app rules	To be tested

The test outcomes must be recorded after execution. Do not mark a test as passed unless the observed output matches the expected behavior.

5.3 AST Output

The parser was previously observed to produce a JSON AST containing five statements for the example program discussed during development. The output contained declarations for age, mark, grade, and passed, followed by an assignment to age. The root node was Program, and the statements were stored in an array.

The declaration nodes contained the variable identifier, declared type, and associated value. The assignment node contained the target identifier and assigned value. Line and column information was also included. The Boolean value was represented as a JSON Boolean, and numeric values were categorized using the NUMBER token type.

This output demonstrates the parser's structured representation of the source program. It does not independently prove that semantic validation was successful.

5.4 Error Handling

Error handling helps users identify invalid source programs. Potential errors include invalid tokens, missing semicolons, incomplete declarations, invalid statement structures, undeclared variables, duplicate declarations, and incompatible data types.

Lexical and syntax errors should be documented based on the actual behavior of the lexer and parser. Semantic errors should be included only if the semantic analyzer implements the relevant checks. For each tested error, record the input, actual error message, and observed result.

5.5 Screenshots

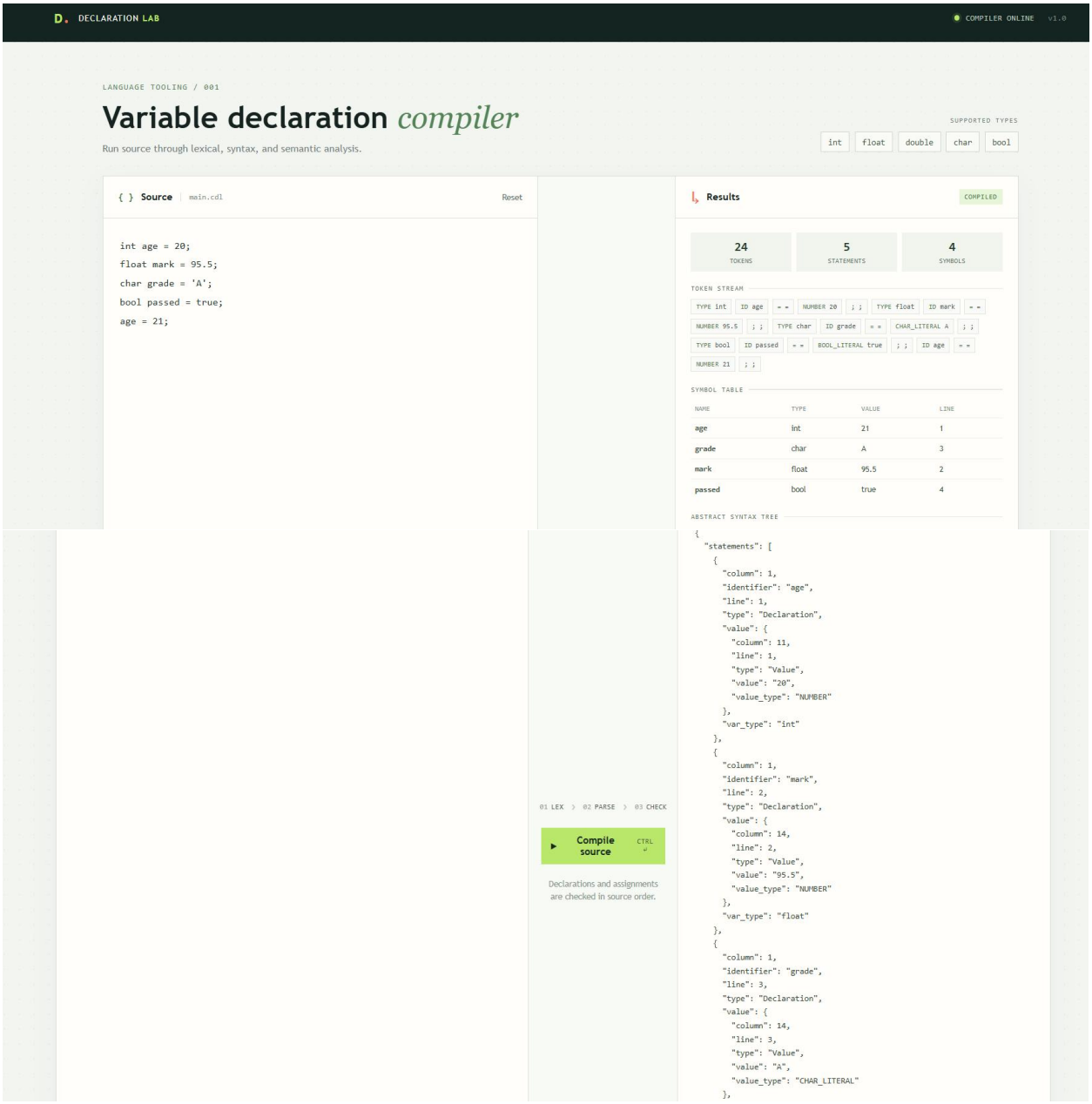


Figure 5.1: Main application interface.

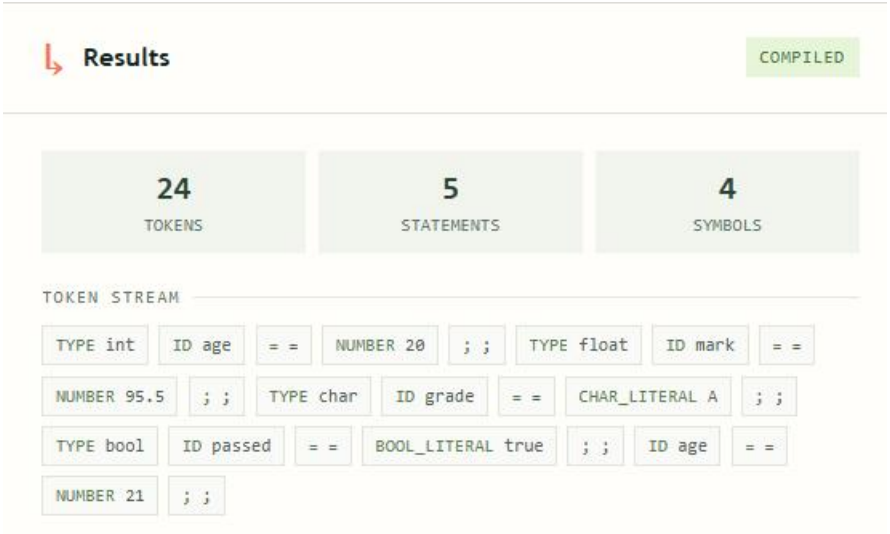


Figure 5.2: Tokens generated by the lexical analyzer.



```
        "value": "A",
        "value_type": "CHAR_LITERAL"
      },
      "var_type": "char"
    },
    {
      "column": 1,
      "identifier": "passed",
      "line": 4,
      "type": "Declaration",
      "value": {
        "column": 15,
        "line": 4,
        "type": "Value",
        "value": true,
        "value_type": "BOOL_LITERAL"
      },
      "var_type": "bool"
    },
    {
      "column": 1,
      "identifier": "age",
      "line": 5,
      "type": "Assignment",
      "value": {
        "column": 7,
        "line": 5,
        "type": "Value",
        "value": "21",
        "value_type": "NUMBER"
      }
    }
  ],
  "type": "Program"
}
```


Figure 5.3: AST output for a valid program.

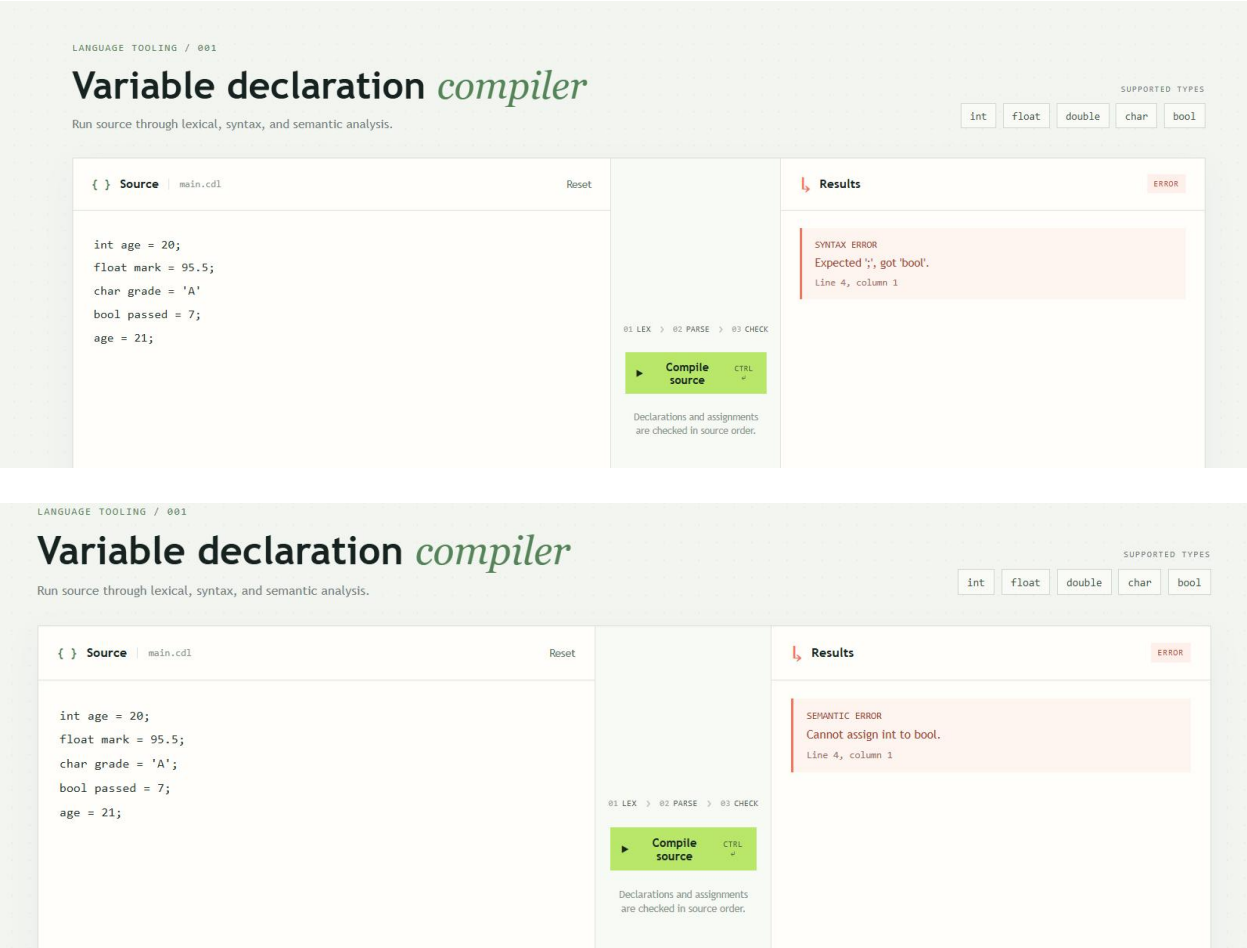


Figure 5.5: Syntax-error output for an invalid program.

CHAPTER 6: PROJECT STRUCTURE AND DOCUMENTATION

6.1 Source Code Structure

The project is organized into modules that separate lexical analysis, grammar definition, parsing, semantic analysis, and application functionality. The following is a provisional directory structure and must be replaced with the actual project structure.

```
cd_project/
    app.py
    lexer.py
    grammar.py
    parser.py
    semantic_analyzer.py
    test_compiler.py
    templates/
        index.html
    static/
        style.css
        script.js
    README.md
```

Some filenames and directories listed above have not been confirmed. Remove any that do not exist in the final project.

6.2 Module Responsibilities

lexer.py: Identifies tokens from the input source code.

grammar.py: Defines the context-free grammar and production rules.

parser.py: Performs syntax analysis and constructs AST nodes.

semantic_analyzer.py: Performs semantic validation, if this is the actual filename and the module is implemented.

app.py: Runs the application backend, if the project uses this entry point.

templates/index.html: Defines the HTML structure of the web interface, if present.

static/script.js: Handles frontend interactions, if present.

static/style.css: Defines the appearance of the web interface, if present.

test_compiler.py: Contains automated tests, if created.

README.md: Documents the project, requirements, setup instructions, and usage.

6.3 README Documentation

The README file provides information about the project and explains how to set up and run it. It should include the project description, supported language constructs, technologies used, directory structure, installation requirements, run instructions, sample programs, expected output, testing instructions, and known limitations.

The project uses app.py is the application entry point, a possible PowerShell command is:

```
python app.py
```

CHAPTER 7: INDIVIDUAL CONTRIBUTIONS

The project involves tasks related to compiler design, implementation, testing, and documentation. The following is a suggested allocation and must be updated to reflect the actual work completed by each team member.

Member	Responsibility	Contribution
Member 1:Sanjith	Lexical Analyzer	Design, implement, or test token recognition.
Member 2:Naveena	Grammar Design,ppt,documentation	Define the CFG and document production rules and grammar properties.
Member 3:Pavan	Parser and AST	Develop parsing logic and verify AST construction.
Member 4:Ronal	Symbol Table	Maintained variable names, data types, and checked duplicate declarations.
Member 5:Nancy	Semantic Analyzer	Implement or test semantic validation, where required
Member 6:Shawn	Integration and Testing	Integrate available modules, test the application, or prepare documentation.

CHAPTER 8: LIMITATIONS AND FUTURE ENHANCEMENTS

8.1 Limitations

The project focuses on a simplified programming language and a limited set of compiler-front-end features. Depending on the final implementation, limitations may include restricted language constructs, lack of function or loop support, limited expression handling, limited error recovery, incomplete semantic checks, and absence of intermediate-code or machine-code generation.

The final report should include only limitations that accurately describe the completed project.

8.2 Future Enhancements

- Arithmetic and logical expressions: Support operators, operands, and precedence rules.
- Additional language constructs: Add conditional statements, loops, and function declarations.
- Improved semantic analysis: Implement more comprehensive checks for declarations, types, and assignments.
- Better error reporting: Provide detailed messages with line and column numbers.
- Graphical AST visualization: Display the AST as a hierarchical tree rather than only as JSON.
- Automated testing: Expand tests to cover valid inputs, invalid syntax, and semantic errors.
- Intermediate-code generation: Extend the compiler to generate an intermediate representation.

These enhancements would increase the language's capabilities and provide opportunities to explore additional stages of compiler construction.

CHAPTER 9: CONCLUSION

The project entitled “Design and Implementation of a Variable Declaration Compiler” demonstrates the fundamental concepts involved in developing a compiler front end for a simplified programming language. It focuses on recognizing tokens, validating program syntax, constructing an Abstract Syntax Tree, and supporting semantic analysis.

The project uses a modular approach to separate the grammar, lexical analyzer, parser, and other compiler components. The AST provides a structured representation of variable declarations and assignments, making the program easier to inspect and process. The JSON representation of the AST also supports debugging and testing.

Through this project, the principles of compiler construction can be studied through practical implementation. The project provides a foundation for further improvements, including additional language constructs, stronger semantic validation, improved error reporting, and intermediate-code generation.